

Write-up, PA2

Student: Margarita Serebrennikova

Student ID: 240137

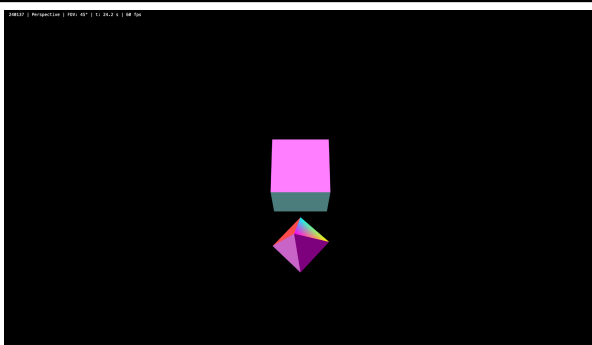
E1

$$\text{solidModelMatrix}(t)M = R_{\text{orbit}}\left(x, \frac{2\pi t}{13}\right) * T(0, 0, 2.5) * R_y(2.0 * t) * S(s(t))$$

(a)

Code change:

```
mat4.translate(m, m, [0, 0, ORBIT_RADIUS]); //was 2nd, now 1st
mat4.rotate(m, m, orbitAngle, ORBIT_AXIS); //was 1st, now 2nd
```



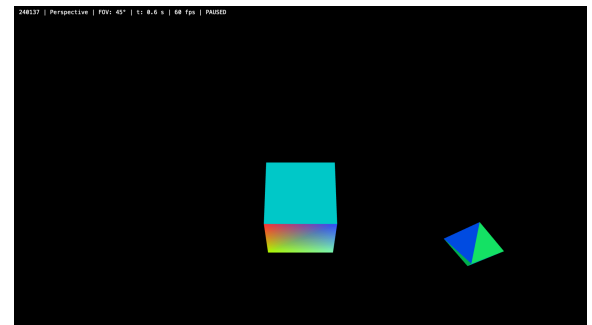
Result:

Swapping translation and rotation places, it gives us $M=T*R_{\text{orbit}}*R_y*S$. As a result, octahedron stays in one place, in front of cube at (0,0,2.5) and falls around x-axis through own center. Each glMatrix call right-multiplies, so the vertex meets the last call first. R_{orbit} now acts before T , while the solid is still at its own origin, so it only turns the solid in place. T is applied last and simply moves it to a fixed spot.

(b)

Code change:

```
mat4.rotate(m, m, orbitAngle, ORBIT_AXIS);
mat4.rotate(m, m, SELF_SPIN_SPEED * t, [0, 1, 0]); // was 3rd, now 2nd
mat4.translate(m, m, [0, 0, ORBIT_RADIUS]); // was 2nd, now 3rd
mat4.scale(m, m, [s, s, s]);
```



Result:

Octahedron no longer spins in place. It's carried around the cube, fast circle around y (2 rad/s), combined with the x-orbit, so it wanders over a sphere of radius 2.5. R_y now acts after the translation, so it rotates the offset (0, 0, 2.5) around the cube's centre instead of the solid around its own centre.

E2

Correct:

(a) the second ... passes [0, 1, 0];

Rotation axis are formatted as [x, y, z], thus, [0, 1, 0] involves Y-axis. To fix what, we should substitute it with [1, 0, 0]

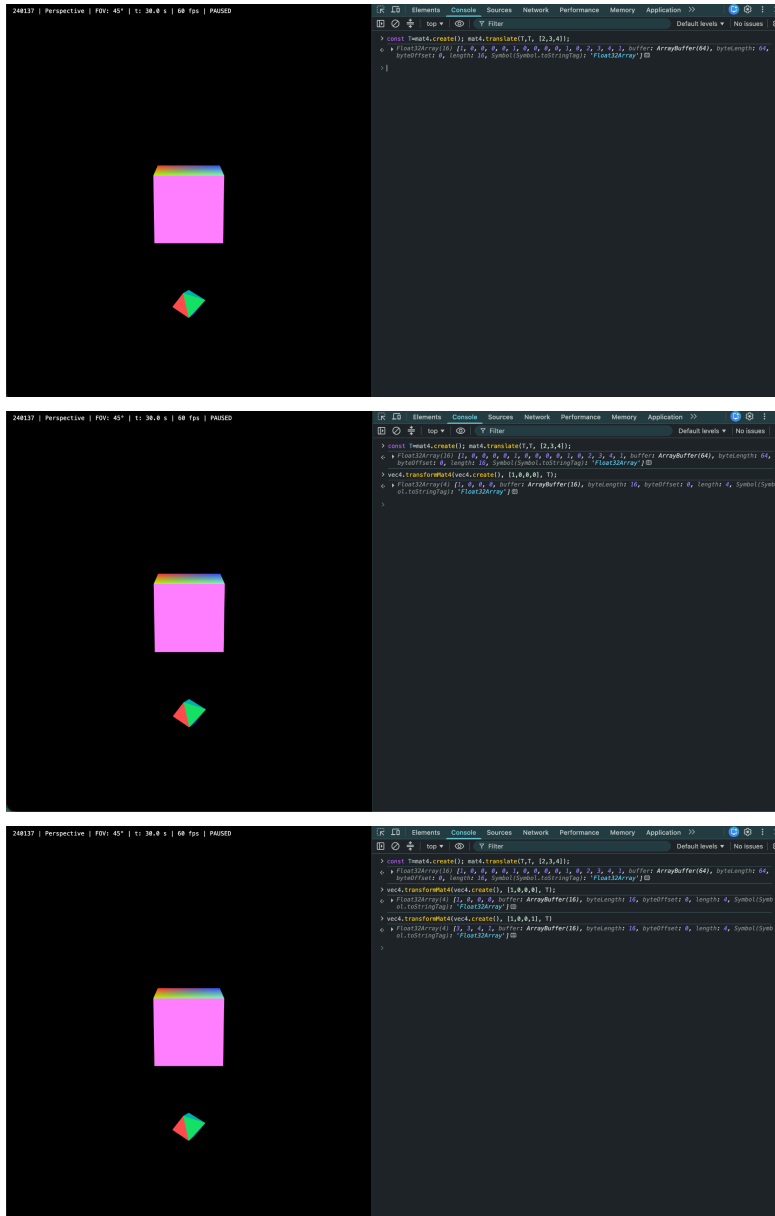
(b) removing ... called "culling"

It is **clipping**. In clipping, each triangle is cut against the six planes of the view volume, and only inside part is kept. Culling is a separate, optional step (`gl.enable(gl.CULL_FACE)`) that discards whole triangles facing away from the camera, and my program does not use it.

(c) the chapter links w to quaternions.

w is the homogeneous coordinate, not part of quaternion. With w = 1 the vector is a point and translation moves it. With w = 0 it's a direction and translation has no effect.

(c), transform



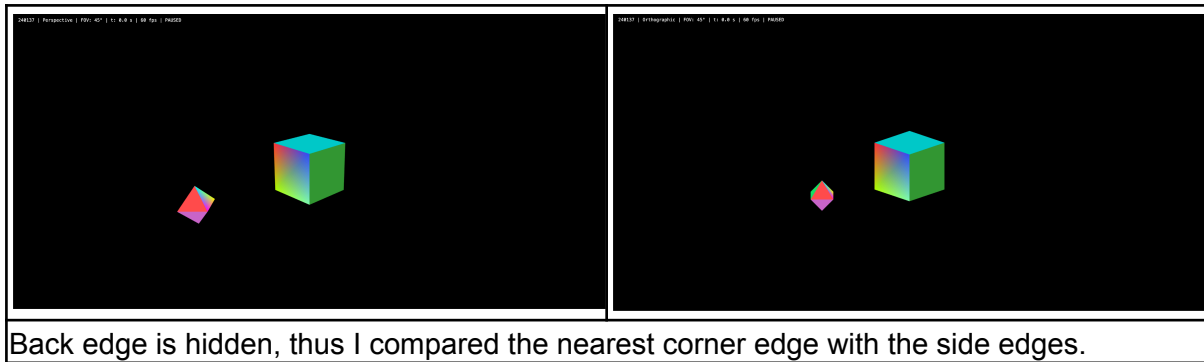
The translation offsets sit in the last column of the matrix (2,3,4 at the end of the column-major array) and are multiplied by w. So the point [1,0,0,1] (w = 1) is moved to [3, 3, 4, 1], while direction [1,0,0,0] (w = 0) stays unchanged.

After the projection matrix, w also holds the vertex's distance from the camera, which the GPU uses for the perspective divide

E3

Prediction: cube's vertical edge would look longer than edges further from screen in perspective. In orthographic, all edges will have same length

T 0.0s, 45° rotation, Perspective	Orthographic
-----------------------------------	--------------



Projection	Middle Edge	Farthest visible edge	Ratio (middle/furthest)
Perspective	251px	232px	1.082
Orthographic	232px	232px	1

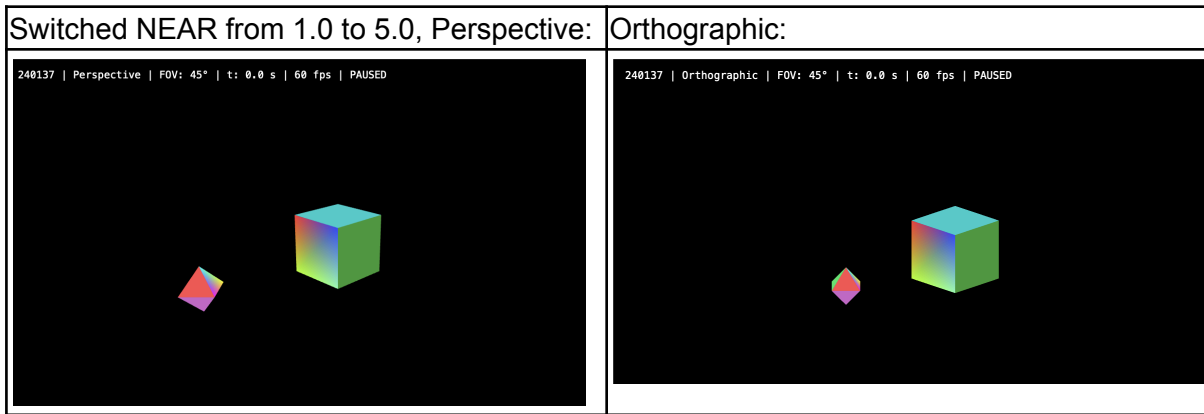
Observation: perspective matrix stores each vertex's distance from the camera in w, and the GPU divides x and y by w, so farther edges appear smaller. The orthographic matrix keeps w = 1, so there is no divide and equal edges stay equal.

E4

Eye is at (0,2.5,7). It is 7.43 units from the cube's centre. Over a full orbit, closest any vertex gets to the eye is about 4.57 units, and the farthest is about 10.33 units. Every vertex must lie between the near and far planes, so I chose NEAR = 1.0 and FAR = 20.0.

(a)

Prediction: when the octahedron is in front of the cube, closer than 5 units, its front part will be cut off.



Cube: In Perspective, top face narrows towards back & side edges look a bit shorter than the middle edge. In orthographic, the top face is like even diamond and all vertical edges are the same length.

Octahedron: bigger and further from cube in perspective and smaller & closer in Orthographic. Orthographic has no divide, so it shows the true size and position.

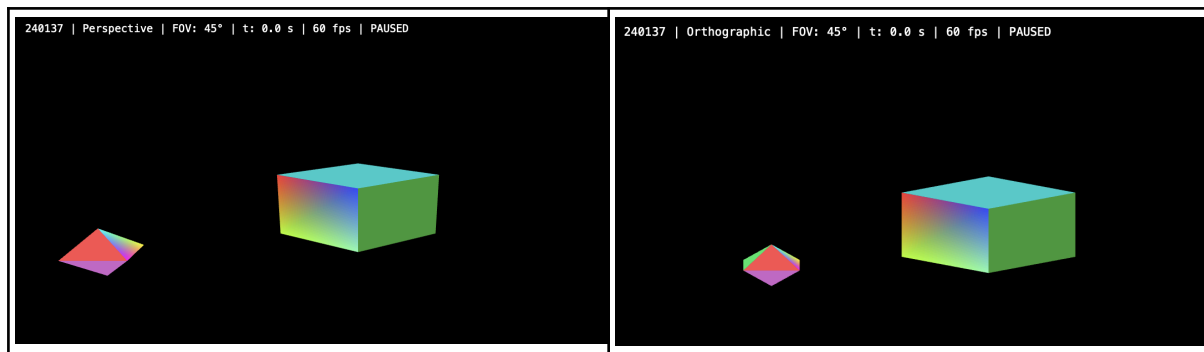
(b)

Changing

```
aspect = canvas.width / canvas.height to aspect =1:
```

Prediction: in wide window, the scene will look stretched sideways

Perspective:	Orthographic:
--------------	---------------

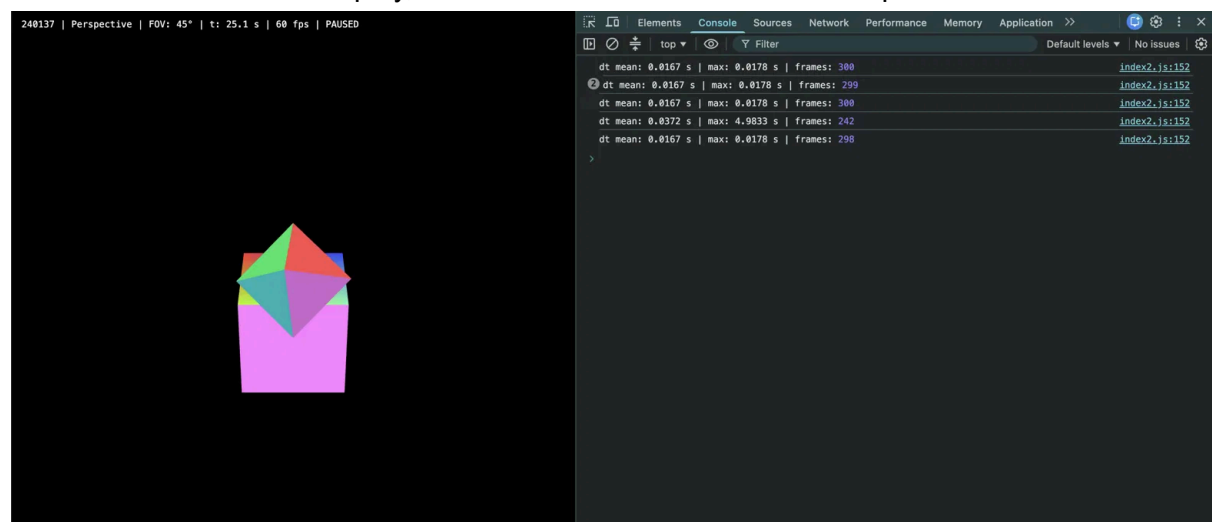


Projection matrix assumes **square representation**, but the viewport displays NDC x within the range $[-1;1]$, so x is distributed across larger number of pixels than y, and everything is stretched horizontally in accordance with the window's width-to-height ratio. Calculating the aspect ratio = `canvas.width / canvas.height` in the `resize()` function allows to adjust the projection so that the cube remains a cube regardless of the window size.

E5

Part 1.

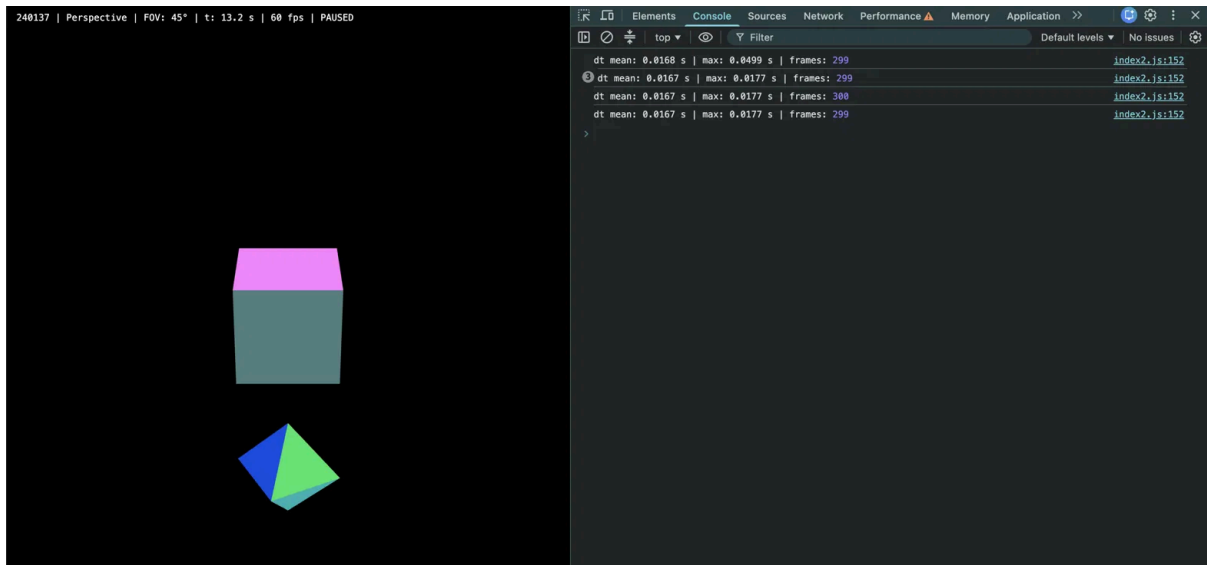
Prediction: on a 60 Hz display, dt should be about $1/60 \approx 0.0167$ s per frame.



Result: I logged dt for 5 s. The mean was 0.0167 s and the maximum 0.0178 s, over about 300 frames, so the scene runs at a steady 60 fps.

Part 2.

Prediction: throttling (4x) may lower the frame rate, but one orbit should still take $T=13$ s, because the motion depends on time, not on frames.

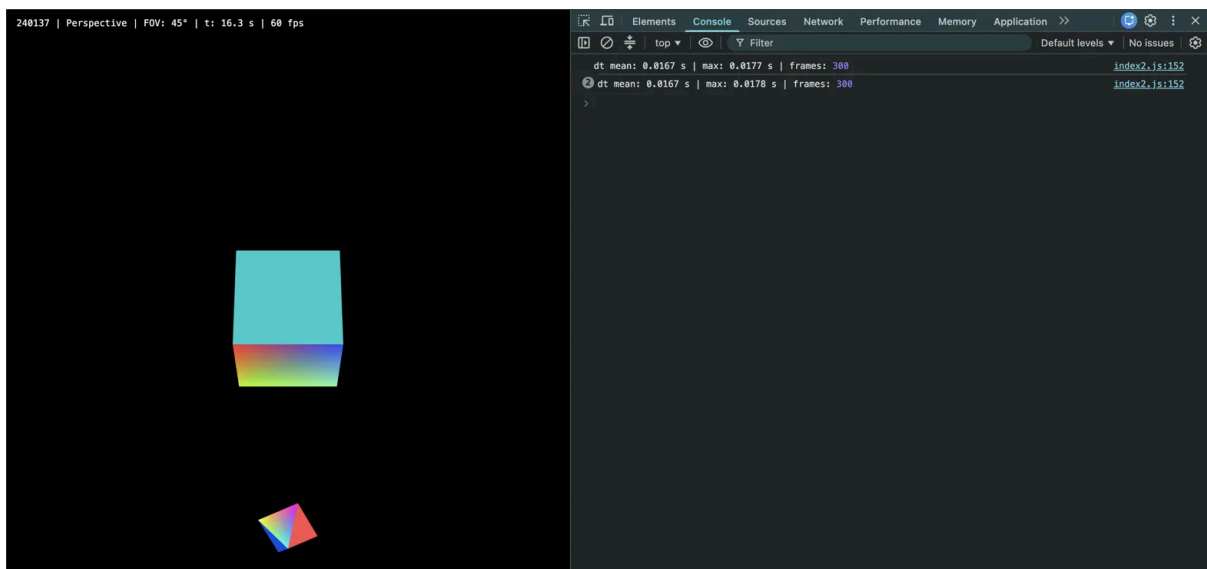


Result: with 4x CPU throttling active, I paused after one orbit by stopwatch. The label showed $t = 13.2$ s, and octahedron was back at its starting position in front of the cube. The scene is light, so the frame rate stayed close to 60 fps (mean dt 0.0167 s, max 0.0499 s).

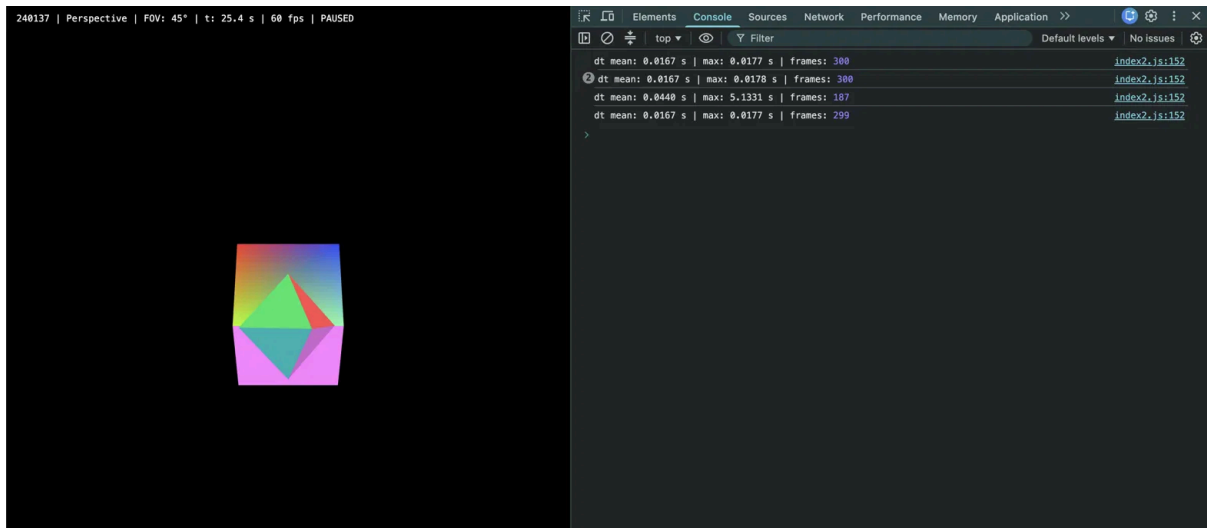
Part 3.

Prediction: without the clamp, the scene will jump forward after returning from another tab.

Result: I set $dt = rawDt$ and switched tabs for about 5s.



Before switching, t was 16.3s.



Right after returning, it was 25.4s, and octahedron moved from below to front of cube. The console shows a maximum dt of 5.13s. The rest is the normal running time before and after the switch.

E6

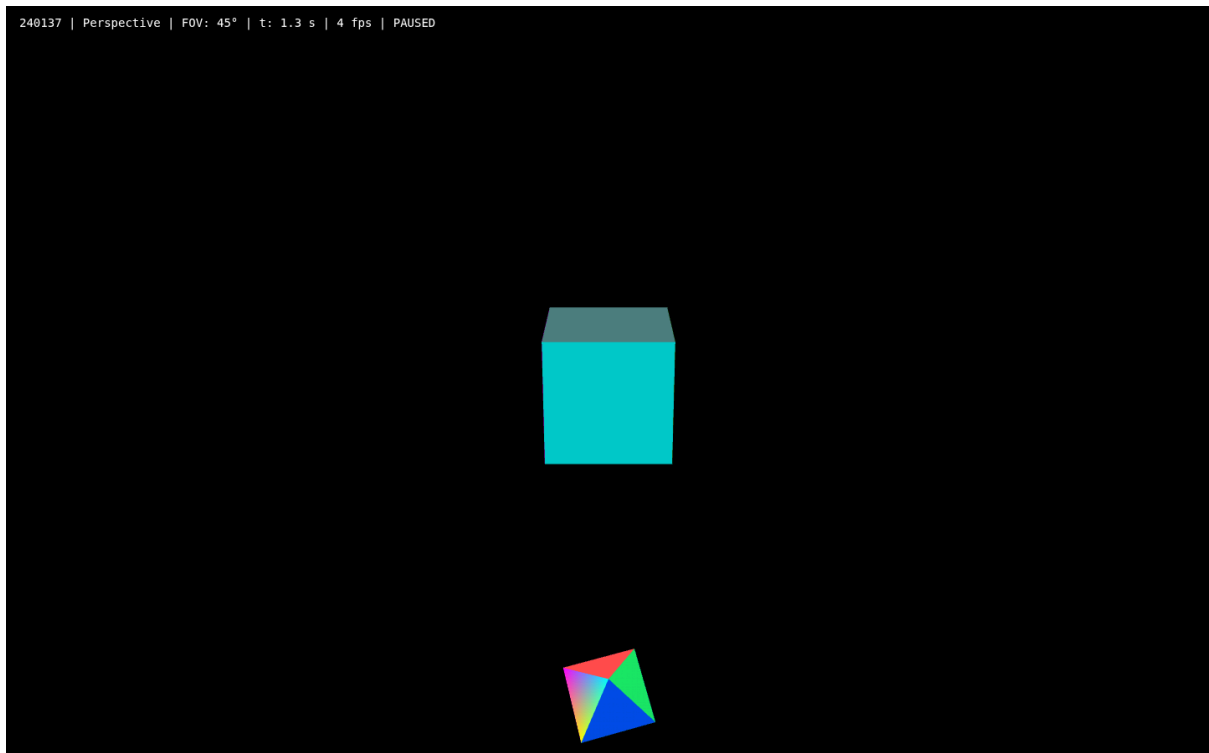
Setup: cube spins around the x-axis at 1.2 rad/s, so at $t=1.309$ s it has turned $1.2 \cdot 1.309 = 1.5708$ rad or $\pi/2 = 90^\circ$. I set $state.t = 1.309$ with *paused: true*

Prediction:

$M=R_x(90^\circ)=\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$	$v= \begin{bmatrix} 0.5, \\ 0.5, \\ -0.5, \\ 1 \end{bmatrix}$
---	---

$x'=1 \cdot 0.5=0.5$; $y'=0 \cdot 0.5 - 1 \cdot (-0.5)=0.5$; $z'=1 \cdot 0.5 + 0 \cdot (-0.5)=0.5$; $w=1$

$M \cdot v=(0.5, 0.5, 0.5, 1)$



Result:

$M \cdot v = (0.5, 0.4999982, 0.5000018, 1)$

Small differences come from 1.309 s being slightly off $\pi/2 \div 1.2$. Full product $P \cdot V \cdot M \cdot v$ gives the clip-space vector (0.7544, 0.7308, 5.4039, 6.7940).

NDC computing: dividing x, y and z by $w = 6.7940$:

$x_ndc = 0.7544 / 6.7940 = 0.111$

$y_ndc = 0.7308 / 6.7940 = 0.108$

$z_ndc = 5.4039 / 6.7940 = 0.795$

3 NDC values are inside $[-1, 1]$, vertex is visible, slightly right of and above the centre. The value $w = 6.794$ is vertex's distance in front of the camera, put there by the perspective matrix. Only x depends on window shape, because the projection divides it by the aspect ratio.

Development Log

Bug: page showed blank screen

browser console showed SyntaxError: missing) after argument list, line 133.

Fix: changed `}};` to `});`. The next console error was ReferenceError: cubecount is not defined, caused by lowercase names (cubecount, solidcount, totalcount) not matching cubeCount and solidCount, so I renamed them consistently.

```
125         state.azimuth += 5 * Math.PI / 180
126         break;
127         case "r": case "R":
128             state.t = 0;
129             state.fovDeg = FOV_START;
130             state.azimuth = 0;
131             break;
132     }
133 });
```

```
37  /*===== Define and Store the Geometry =====*/
38  const cube=buildCube();
39  const solid=buildOctahedron();
40  const positions=cube.positions.concat(solid.positions);
41  const colors=cube.colors.concat(solid.colors);
42  const cubeCount=cube.positions.length/3;
43  const solidCount=solid.positions.length/3;
44  const totalCount=cubeCount+solidCount;
```